

1. Minimax Search
2. Alpha-Beta Pruning
3. Heuristic Alpha-Beta Search
4. Monte Carlo Tree Search (MCTS)

Here i am using Tic-Tac-Toe as the game environment

1)

```
g++ minimax.cpp -o minimax
./minimax
```

```
Best Move for X: 5
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 %
```

2)

```
g++ -std=c++11 alphabeta.cpp -o alphabeta
./alphabeta
```

```
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % g++ -std=c++11 alphabeta.cpp -o alphabeta
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % ./alphabeta
Best Move: 5
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 %
```

3)

```
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % g++ -std=c++11 heuristic_alphabeta.cpp -o heuristic
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % ./heuristic
Best Move: 8
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 %
```

4)

```
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % g++ -std=c++11 montecarlo_treearch.cpp -o mcts
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 % ./mcts
Best Move using MCTS: 8
(base) rangoju@Rangojus-MacBook-Pro ai_assignment5 %
```

Test Case 1

```
X O X
O O _
X _ _
```

Expected:

AI blocks or wins correctly.

Test Case 2

```
X _ _
_ O _
_ _ X
```

Expected:

AI chooses strategic corner/center moves.

Complexity:

Algorithm	Time Complexity	Improvement
Minimax	$O(b^d)$	Full search
Alpha-Beta	$O(b^{(d/2)})$ best case	Pruning
Heuristic Alpha-Beta	Limited depth	Faster
MCTS	Probabilistic	Scales better

Where:

b = branching factor

d = search depth